Excellence in Learning Innovation

LAPORAN PRAKTIKUM PROGRES 1
IMPLEMENTASI ARENA ALLOCATOR
UNTUK STRUKTUR DATA BERBASIS SISTEM

Disusun untuk

Memenuhi Tugas Mata kuliah struktur data yang diampu oleh

Bapak Muhammad Jauharul Fuady, S.T., M.T.

Disusun Oleh:

Ahtaullah Akmal Irianto	(250535624584)
Al Zahra Ramadhani	(250535623767)
Fariz Julian Nugraha	(250535623884)

PROGRAM STUDI S1 TEKNIK INFORMATIKA

DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS NEGERI MALANG

2026

1. PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan sistem perangkat lunak, efisiensi penggunaan memori merupakan salah satu parameter krusial yang menentukan performa aplikasi. Secara umum, manajemen memori sering kali mengandalkan fungsi alokasi konvensional seperti malloc dan free. Namun, pada skenario sistem yang membutuhkan alokasi objek dalam jumlah besar dengan waktu hidup yang serupa, mekanisme konvensional ini sering kali menyebabkan fragmentasi memori dan penurunan performa akibat *overhead* pemanggilan sistem yang berulang.

Sebagai solusi atas permasalahan tersebut, tugas ini berfokus pada implementasi Arena Allocator. Arena Allocator bekerja dengan mengalokasikan satu blok memori besar di awal, yang kemudian dikelola secara mandiri untuk kebutuhan berbagai struktur data. Melalui pendekatan ini, diharapkan mahasiswa dapat memahami lebih dalam mengenai manajemen memori tingkat rendah, konsep *cache locality*, serta bagaimana representasi data seperti *pointer* dapat diwujudkan dalam bentuk *offset* integer.

1.2 Tujuan Penugasan

Berdasarkan deskripsi penugasan yang diberikan, tujuan dari proyek ini adalah:

1. Mengimplementasikan *Arena Allocator* sebagai *Abstract Data Type* (ADT) yang mandiri tanpa bergantung pada alokasi memori dinamis di level struktur data.
2. Membangun struktur data (seperti *array* dan *linked list*) yang seluruh elemennya berada di dalam satu blok memori terpusat.
3. Melakukan analisis komparatif antara penggunaan *Arena Allocator* dengan mekanisme malloc/free dari sisi performa dan efisiensi memori.
4. Memvisualisasikan tata letak memori dalam bentuk grid ASCII untuk memahami perilaku alokasi secara riil.

1.3 Lingkup Tahapan Minggu ke-1

Pada tahap awal ini (Minggu 1), fokus utama tim adalah membangun fondasi sistem berupa *Arena Allocator*. Implementasi difokuskan pada pembuatan fungsi-fungsi dasar API arena yang meliputi:

1) arena_init / arena_create: Inisialisasi blok memori awal.
2) arena_alloc: Mekanisme pembagian memori di dalam arena.
3) arena_get: Pengambilan data berdasarkan *offset*.
4) arena_reset: Pengosongan arena secara instan dengan kompleksitas

2. IMPLEMENTASI PROGRAM

Kita mempunyai konsep untuk menerapkan sebuah struktur data Queue -> Batch Processing untuk mengelola sebuah konsep logika sebuah antrean printer. Dimana tahap pertamanya, program akan membuat arena sebagai blok memori utama untuk antrean tersebut. Tetapi, sebelum kita mengimplementasikan struktur data queue pada projek besar kami. Kita harus menyiapkan 5 fungsi krusial untuk membuat arena seperti:

2.1 Arena.h

```
#ifndef ARENA_H
#define ARENA_H

#include <stdio.h>

typedef struct {
    unsigned char *buffer;
    size_t capacity;
    size_t offset;
} Arena;

void arena_init(Arena *arena, size_t size);
int arena_alloc(Arena *arena, size_t size);
void* arena_get(Arena *arena, int offset);
void arena_reset(Arena *arena);
void arena_dump(Arena *arena, int width);

#endif
```

2.2 Arena.c

```
#include <stdio.h>
#include <stdlib.h>
```

```c
#include "arena.h"

void arena_init(Arena *arena, size_t size) {
    arena->buffer = (unsigned char*) malloc(size);
    arena->capacity = size;
    arena->offset = 0;
}

int arena_alloc(Arena *arena, size_t size) {
    if (arena->offset + size > arena->capacity) {
        return -1; // gagal
    }

    int allocated_offset = arena->offset;
    arena->offset += size;

    return allocated_offset;
}

void* arena_get(Arena *arena, int offset) {
    if (offset < 0 || offset >= arena->capacity) {
        return NULL;
    }

    return (void*)(arena->buffer + offset);
}

void arena_reset(Arena *arena) {
    arena->offset = 0;
}

void arena_dump(Arena *arena, int width) {
    printf("\n=== ARENA DUMP ===\n");
```

```
    for (size_t i = 0; i < arena->capacity; i++) {
        if (i % width == 0) printf("\n");

        if (i < arena->offset)
            printf("[#]");
        else
            printf("[.]");
    }

    printf("\n\nOffset: %zu / %zu\n",
arena->offset, arena->capacity);
}
```

2.3 Main.c

```
#include <stdio.h>
#include "arena.h"

int main() {
    Arena arena;
    arena_init(&arena, 32);

    printf("Kondisi Awal:\n");
    arena_dump(&arena, 8);

    int a = arena_alloc(&arena, 5);
    int b = arena_alloc(&arena, 10);

    printf("\nSetelah Alokasi:\n");
    arena_dump(&arena, 8);

    // isi data
```

```
    char *dataA = (char*) arena_get(&arena, a);
    for (int i = 0; i < 5; i++) dataA[i] = 'A';

    char *dataB = (char*) arena_get(&arena, b);
    for (int i = 0; i < 10; i++) dataB[i] = 'B';

    arena_reset(&arena);

    printf("\nSetelah reset:\n");
    arena_dump(&arena, 8);

    return 0;
}
```

3. OUTPUT PROGRAM

```
Kondisi Awal:

=== ARENA DUMP ===

[.][.][.][.][.][.][.][.]
[.][.][.][.][.][.][.][.]
[.][.][.][.][.][.][.][.]
[.][.][.][.][.][.][.][.]

Offset: 0 / 32

Setelah Alokasi:

=== ARENA DUMP ===

[#][#][#][#][#][#][#][#]
[#][#][#][#][#][#][#][.]
[.][.][.][.][.][.][.][.]
[.][.][.][.][.][.][.][.]

Offset: 15 / 32

Setelah reset:

=== ARENA DUMP ===

[.][.][.][.][.][.][.][.]
[.][.][.][.][.][.][.][.]
[.][.][.][.][.][.][.][.]
[.][.][.][.][.][.][.][.]

Offset: 0 / 32
```

4. PEMBAHASAN DAN KESIMPULAN

4.1 Penjelasan

Program praktikum ini mengimplementasikan sebuah **arena allocator** dalam bahasa C, yaitu sebuah teknik manajemen memori di mana satu blok memori besar dialokasikan sekaligus, kemudian digunakan secara bertahap dengan cara menggeser sebuah penanda posisi (*offset*). Program terdiri dari tiga file, yaitu `arena.h` sebagai header, `arena.c` sebagai implementasi, dan `main.c` sebagai program utama untuk pengujian.

4.1.1 File `arena.h`

File header ini berisi deklarasi struktur data dan fungsi-fungsi yang akan digunakan. Struktur `Arena` didefinisikan dengan tiga komponen utama, yaitu `buffer` yang bertipe `unsigned char*` dan berfungsi sebagai tempat penyimpanan data mentah, `capacity` yang bertipe `size_t` dan menyimpan ukuran total arena, serta `offset` yang juga bertipe `size_t` dan berfungsi sebagai penanda posisi kursor alokasi saat ini. Penggunaan tipe `unsigned char*` pada `buffer` dipilih karena ukurannya dijamin satu byte, sehingga memudahkan operasi aritmetika pointer per byte.

Pada header ini juga dideklarasikan lima fungsi utama, yaitu `arena_init` untuk inisialisasi, `arena_alloc` untuk alokasi, `arena_get` untuk mengambil pointer dari offset, `arena_reset` untuk mengosongkan arena, dan `arena_dump` untuk menampilkan kondisi arena secara visual.

4.1.2 File `arena.c`

File ini berisi implementasi dari fungsi-fungsi yang telah dideklarasikan pada header. Fungsi `arena_init` bertugas menginisialisasi arena dengan meminta blok memori ke sistem operasi melalui fungsi `malloc` sebesar parameter yang diberikan. Setelah blok memori diperoleh, kapasitas arena diatur sesuai ukuran yang diminta, dan offset diatur ke nol sebagai posisi awal. Perlu dicatat bahwa setelah inisialisasi ini, semua alokasi berikutnya tidak lagi

berinteraksi dengan sistem operasi, melainkan hanya memanipulasi blok memori yang sudah diperoleh.

Fungsi `arena_alloc` merupakan inti dari mekanisme bump allocator. Fungsi ini pertama-tama memeriksa apakah jumlah offset saat ini ditambah ukuran yang diminta melebihi kapasitas arena. Jika melebihi, fungsi akan mengembalikan nilai -1 sebagai penanda kegagalan alokasi. Jika masih mencukupi, nilai offset saat ini disimpan ke dalam variabel `allocated_offset`, kemudian offset digeser maju sebesar ukuran yang dialokasikan. Nilai `allocated_offset` tersebut kemudian dikembalikan sebagai hasil. Perlu ditekankan bahwa yang dikembalikan adalah nilai offset berupa integer, bukan pointer. Pendekatan ini dipilih karena offset bersifat stabil dan tidak akan menjadi tidak valid meskipun buffer dipindahkan.

Fungsi `arena_get` berperan sebagai penerjemah dari nilai offset menjadi pointer memori yang sesungguhnya. Fungsi ini melakukan validasi terhadap nilai offset, yaitu memastikan bahwa offset tidak negatif dan tidak melebihi kapasitas arena. Jika valid, fungsi mengembalikan pointer hasil penjumlahan alamat buffer dengan nilai offset. Inilah alasan mengapa proses alokasi dan pengaksesan data dipisahkan menjadi dua langkah, yaitu `arena_alloc` memesan ruang dan `arena_get` memberikan alamat nyata saat data akan digunakan.

Fungsi `arena_reset` cukup sederhana, yaitu hanya mengatur ulang nilai offset menjadi nol. Dengan demikian, seluruh area dalam arena dianggap kosong dan siap digunakan kembali. Data lama secara fisik masih ada dalam buffer, namun akan tertimpa pada alokasi berikutnya. Proses reset ini sangat murah secara komputasi karena tidak melibatkan pembebasan memori secara aktual.

Fungsi `arena_dump` merupakan fungsi bantu untuk debugging yang menampilkan kondisi arena secara visual. Fungsi ini melakukan iterasi sepanjang kapasitas arena dan mencetak simbol `[#]` untuk byte yang telah terpakai dan `[.]` untuk byte yang masih kosong. Parameter `width` digunakan

untuk mengatur berapa banyak byte yang ditampilkan per baris agar output mudah dibaca.

4.1.3 File `main.c`

File ini berfungsi sebagai program utama untuk menguji implementasi arena allocator. Program diawali dengan deklarasi variabel `arena` bertipe `Arena`, kemudian diinisialisasi dengan kapasitas 32 byte. Kondisi awal arena ditampilkan menggunakan `arena_dump` dengan lebar 8 byte per baris, sehingga akan terlihat 32 tanda `[.]` yang menandakan seluruh arena masih kosong.

Selanjutnya, program melakukan dua alokasi berturut-turut, yaitu sebesar 5 byte yang hasil offsetnya disimpan pada variabel `a`, dan 10 byte yang hasil offsetnya disimpan pada variabel `b`. Setelah alokasi ini, offset arena akan berada pada posisi 15, dengan variabel `a` bernilai 0 dan variabel `b` bernilai 5. Kondisi arena kemudian ditampilkan kembali untuk memperlihatkan perubahannya.

Program kemudian mengisi data pada area yang telah dialokasikan. Untuk area pertama, fungsi `arena_get` dipanggil dengan offset `a`, dan hasilnya dikonversi menjadi pointer `char*` . Melalui pointer tersebut, lima byte pertama diisi dengan karakter 'A'. Proses serupa dilakukan untuk area kedua, yaitu sepuluh byte berikutnya diisi dengan karakter 'B'. Terakhir, program memanggil `arena_reset` untuk mengosongkan arena, lalu menampilkan kondisi akhir yang memperlihatkan bahwa seluruh arena kembali seperti kondisi awal.

4.2 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan beberapa hal sebagai berikut. Pertama, arena allocator merupakan teknik manajemen memori yang efisien karena proses alokasi hanya memerlukan operasi penjumlahan sederhana pada offset, berbeda dengan alokasi tradisional

menggunakan `malloc` yang memerlukan pencarian blok memori yang sesuai. Kedua, teknik ini juga bebas dari masalah fragmentasi memori karena seluruh alokasi dilakukan secara berurutan dalam satu blok yang kontinu. Ketiga, konsekuensi dari pendekatan ini adalah hilangnya kemampuan untuk membebaskan objek secara individual, sehingga seluruh arena hanya dapat di-reset atau dihancurkan secara keseluruhan.

Praktikum ini juga menunjukkan bahwa pemisahan antara konsep *offset* dan *pointer* melalui fungsi `arena_alloc` dan `arena_get` memberikan keuntungan tambahan berupa stabilitas referensi, di mana offset tetap valid meskipun buffer mengalami perubahan alamat. Selain itu, operasi `arena_reset` menunjukkan efisiensi arena allocator dalam mendaur ulang memori, yaitu cukup dengan satu instruksi penugasan sederhana tanpa perlu membebaskan memori ke sistem operasi.

Secara keseluruhan, implementasi arena allocator ini berhasil mendemonstrasikan konsep inti dari manajemen memori berbasis region, yaitu satu blok memori besar yang dikelola dengan penanda posisi yang bergeser maju. Teknik seperti ini banyak digunakan dalam situasi di mana banyak objek memiliki siklus hidup yang sama, seperti pada data per-frame dalam game, node AST dalam proses kompilasi, serta penanganan request pada server. Dengan demikian, praktikum ini memberikan pemahaman yang baik tentang alternatif strategi alokasi memori selain pendekatan konvensional menggunakan `malloc` dan `free`